

Page not found - GitHub · GitHub

Two Sum - LeetCode

Running Sum of 1d Array - LeetCode

Settings - LeetCode

ChatGPT

Ask Gemini

leetcode.com/problems/running-sum-of-1d-array/

Problem List

Run

Submit

1480. Running Sum of 1d Array

Easy

Topics

Companies

Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`

Output: `[1,2,3,4,5]`

Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`

Output: `[3,4,6,16,17]`

Constraints:

- `1 <= nums.length <= 1000`
- `-10^6 <= nums[i] <= 10^6`

UNLOCK EXPERIENCES

Prepare your network for today and tomorrow

HPE

8.9K

221

131 Online

Code

Java

Auto

1

2

3

4

5

6

7

8

class Solution {

public int[] runningSum(int[] nums) {

for (int i = 1; i < nums.length; i++) {

nums[i] += nums[i - 1];

}

return nums;

}

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

[1,2,3,4]

Output

[1,3,6,10]

Expected

[1,3,6,10]

Contribute a testcase

33°C

Mostly cloudy

Search

ENG US

07-08-2025

11:16

Problem List

Description | Editorial | Solutions | Submissions

49. Group Anagrams

Medium | Topics | Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- `1 <= strs.length <= 104`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.

22.4K | 425 | 466 Online

Code

Java | Auto

```
3 class Solution {
4     public List<List<String>> groupAnagrams(String[] strs) {
5         Map<String, List<String>> map = new HashMap<>();
6
7         for (String str : strs) {
8             char[] chars = str.toCharArray();
9             Arrays.sort(chars);
10            String key = new String(chars);
11
12            map.putIfAbsent(key, new ArrayList<>());
13            map.get(key).add(str);
14        }
15
16        return new ArrayList<>(map.values());
17    }
18 }
```

Saved | Ln 18, Col 2

Testcase | Test Result

Accepted | Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

strs =
["eat","tea","tan","ate","nat","bat"]

Output

[["eat","tea","ate"],["bat"],["tan","nat"]]

Expected

[["bat"],["nat","tan"],["ate","eat","tea"]]

Contribute a testcase

31°C | Mostly cloudy

Search

ENG US | 12:30 | 07-08-2026

Problem List

DescriptionEditorialSolutionsSubmissions

1732. Find the Highest Altitude

EasyTopicsCompaniesHint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length `n` where `gain[i]` is the net gain in altitude between points `i` and `i + 1` for all $(0 \leq i < n)$. Return the *highest altitude* of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$

Discover moreCalculators & Reference Tools

Seen this question in a real interview before? 1/6

3.5K42739 Online

Code

JavaAuto

```
1 class Solution {
2     public int largestAltitude(int[] gain) {
3         int altitude = 0;
4         int maxAltitude = 0;
5
6         for (int g : gain) {
7             altitude += g;
8             maxAltitude = Math.max(maxAltitude, altitude);
9         }
10
11         return maxAltitude;
12     }
13 }
14
```

SavedLn 14, Col 1

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

1

Contribute a testcase

33°C
Mostly cloudy

Search

ENG
US

12:21
07-08-2025

53. Maximum Subarray

Medium Topics Companies

Given an integer array `nums`, find the **subarray** with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

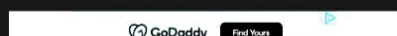
Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the `O(n)` solution, try coding another solution using the **divide and conquer** approach, which is more subtle.



38.2K 572 538 Online

31°C Mostly cloudy



ENG US 12:18 07-08-2025

Code

Java Auto

```
1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int maxSum = nums[0];
4         int currentSum = nums[0];
5
6         for (int i = 1; i < nums.length; i++) {
7             currentSum = Math.max(nums[i], currentSum + nums[i]);
8             maxSum = Math.max(maxSum, currentSum);
9         }
10
11         return maxSum;
12     }
13 }
```

Saved

Ln 13, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =`
`[-2,1,-3,4,-1,2,1,-5,4]`

Output

6

Expected

6

Contribute a testcase

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements must be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`
Output: `2`, `nums = [1,2,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`
Output: `5`, `nums = [0,1,2,3,4,_,_,_,_,_,_]`

19.5K 1.1K 635 Online

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         if (nums.length == 0) {
4             return 0;
5         }
6
7         int k = 1;
8
9         for (int i = 1; i < nums.length; i++) {
10             if (nums[i] != nums[k - 1]) {
11                 nums[k] = nums[i];
12                 k++;
13             }
14         }
15
16         return k;
17     }
18 }
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

[1,2]

Contribute a testcase

Problem List

27. Remove Element

Description

Editorial

Solutions

Submissions

Run

Ctrl

+

27. Remove Element

Easy

Topics

Companies

Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return the *number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [3,2,2,3], val = 3`
Output: `2, nums = [2,2,...]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being `2`. It does not matter what you leave beyond the returned `k` (hence they are underscores).

5.3K

983

345 Online

Code

Java

Auto

```
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k = 0;
4
5         for (int i = 0; i < nums.length; i++) {
6             if (nums[i] != val) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11
12        return k;
13    }
14 }
```

Saved

Ln 14, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =

[3,2,2,3]

val =

3

Output

[2,2]

Expected

[2,2]

Problem List

1470. Shuffle the Array

EasyTopicsCompaniesHint

Given the array `nums` consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.
Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: `nums = [2,5,1,3,4,7], n = 3`

Output: `[2,3,5,4,1,7]`

Explanation: Since $x_1=2, x_2=5, x_3=1, y_1=3, y_2=4, y_3=7$ then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1], n = 4`

Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2], n = 2`

Output: `[1,2,1,2]`

Constraints:

$1 \leq n \leq 500$

`nums.length == 2n`

$1 \leq \text{nums}[i] \leq 10^3$

6.5K

166

56 Online

Code

JavaAuto

```
1 class Solution {
2     public int[] shuffle(int[] nums, int n) {
3         int[] result = new int[2 * n];
4         int index = 0;
5
6         for (int i = 0; i < n; i++) {
7             result[index++] = nums[i];
8             result[index++] = nums[i + n];
9         }
10
11         return result;
12     }
13 }
```

Testcase

Test Result

AcceptedRuntime: 0 ms

Case 1

Case 2

Case 3

Input

`nums =`

`[2,5,1,3,4,7]`

`n =`

`3`

Output

`[2,3,5,4,1,7]`

Expected

`[2,3,5,4,1,7]`

Problem List

49 Online

Description

Editorial

Solutions

Submissions

Easy

Topics

Companies

Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the *wealth* that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`
Output: 6
Explanation:
1st customer has wealth = 1 + 2 + 3 = 6
2nd customer has wealth = 3 + 2 + 1 = 6
Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: `accounts = [[1,5],[7,3],[3,5]]`
Output: 10
Explanation:
1st customer has wealth = 6
2nd customer has wealth = 10
3rd customer has wealth = 8
The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: `accounts = [[2,8,7],[7,1,3],[1,9,5]]`
Output: 17

Constraints:

- $m == \text{accounts.length}$
- $n == \text{accounts}[i].\text{length}$
- $1 \leq m, n \leq 50$

Code

Java

Auto

```
1 class Solution {
2     public int maximumWealth(int[][] accounts) {
3         int maxWealth = 0;
4
5         for (int i = 0; i < accounts.length; i++) {
6             int wealth = 0;
7             for (int j = 0; j < accounts[i].length; j++) {
8                 wealth += accounts[i][j];
9             }
10            maxWealth = Math.max(maxWealth, wealth);
11        }
12
13        return maxWealth;
14    }
15 }
```

SavedLn 15, Col 2

Testcase

Test Result

AcceptedRuntime: 0 ms

Case 1

Case 2

Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

Output

6

Expected

6

Contribute a testcase